SK하이닉스 AI 해커톤 2026 대상 수상을 위한 심층 리서치 및 전략 보고서

1. SK하이닉스 및 엔터프라이즈 AI 현장 니즈 분석

글로벌 인공지능 패권 경쟁이 심화됨에 따라 반도체 산업의 핵심 화두는 단순한 연산 장치의 속도 개선을 넘어 데이터의 흐름 전체를 설계하는 메모리 중심의 AI 인프라스트럭처로 이동하고 있다[1]. 이러한 거시적 변화 속에서 SK하이닉스는 HBM(고대역폭 메모리), AI-DRAM, eSSD로 이어지는 계층화된 메모리 포트폴리오를 통해 AI 워크로드의 병목 현상인 '메모리 월(Memory Wall)'을 극복하는 데 전사적 역량을 집중하고 있다[1]. 그러나 이러한 차세대 반도체의 양산 및 관리 과정에서는 고도화된 공정 미세화로 인해 과거에 경험하지 못한 수준의 수율(Yield) 관리 및 인프라 자동화 문제가 발생하고 있으며, 이는 글로벌 해커톤에서 기업 심사위원들이 가장 갈증을 느끼는 핵심 페인포인트(Pain Point)로 작용한다.

SK하이닉스의 HBM 제조 공정 및 현장 페인포인트 심층 해부

현재 SK하이닉스의 가장 중대한 기술적 과제 중 하나는 12단 및 16단을 넘어 20단 이상으로 적층(Stacking)되는 HBM4 및 HBM5 제품군의 패키징 안정성과 수율 플로어(Yield floor)를 확보하는 것이다[3]. 칩을 수직으로 높게 쌓아 올릴수록 방열(Heat dissipation)과 패키징 복잡성이 기하급수적으로 증가하며, 물리적 제약을 극복하기 위해 마이크로 범프 없이 구리와 구리를 직접 결합하는 하이브리드 본딩(Hybrid Bonding) 기술의 도입이 필수적으로 요구된다[3]. SK하이닉스 경영진은 이 기술을 실제 양산에 적용하기 위한 타협 불가능한 전제 조건으로 '90~98% 수준의 결함 없는 수율'과 '3~5년의 수명'을 명시한 바 있다[3].

이러한 극한의 수율 목표를 달성하기 위해 제조 라인(Fab)에서는 FDC(Fault Detection and Classification, 설비 이상 감지 및 분류)와 같은 고도화된 통계적 공정 제어 시스템을 운영한다[4]. 그러나 현장 엔지니어와 데이터 과학자들이 직면한 실제 병목 현상은 설비 센서에서 폭발적으로 쏟아지는 시계열 데이터의 파편화와 맥락(Context) 부재에 있다. 팹 내의 수많은 공정 장비는 SEMI 표준인 SECS/GEM(반도체 장비 통신 표준) 및 HSMS(High-Speed SECS Message Services) 프로토콜을 통해 초당 수만 건의 온도, 압력, 가스 유량 데이터를 쏟아낸다[6]. 문제는 이러한 원시 데이터(Raw data) 단독으로는 의미를 가지지 못하며, 해당 시점에 챔버(Chamber) 내에 어떤 레시피(Recipe)가 적용되었고, 어떤 웨이퍼가 처리 중이었는지에 대한 다차원적인 맥락이 결합되어야만 비로소 이상 원인을 규명할 수 있다는 점이다[5].

현재 FDC 알람이 발생하거나 웨이퍼 검사 장비(KLA Surfscan 등)에서 결함이 한계치를 초과(Out Of Control, OOC)할 경우, 엔지니어가 수동으로 SECS/GEM 로그를 추출하고 웨이퍼 결함 이미지를 대조하며 8D 리포트(문제 해결 보고서)를 작성하는 데 수 시간에서 수일이 소요된다[8]. 24시간 가동되어야 하는 반도체 공장에서 이러한 분석 지연(Latency)은 곧바로 설비 종합 효율(OEE, Overall Equipment Effectiveness)의 심각한 저하와 천문학적인 비용 손실로 직결된다[11]. 또한, HBM 테스트 장비와 소프트웨어 파이프라인(MLOps/DevOps)을 관리하는 스크립트 모듈에서 간헐적인 런타임 오류가 발생할 때마다 엔지니어의 수동 개입이 요구되는 구조적 한계 역시 시급히 해결해야 할 소프트웨어 엔지니어링 병목으로 지목된다[6].

엔터프라이즈 레벨의 AI 도입 요구사항

단순한 기술적 시연을 넘어 실제 사내망에 도입할 수 있는 수준의 솔루션을 갈망하는 기업 기술

임원(심사위원)들의 요구사항은 매우 보수적이고 엄격하다. 생성형 AI가 가진 태생적 불안정성을 통제하고 프로덕션 환경에 배포하기 위해서는 다음과 같은 핵심 조건들이 충족되어야 한다.

엔터프라이즈 요구사항	현장의 제약 조건 및 심사위원의 우려 사항	기술적 해결 및 아키텍처 방안
보안성 및 온프레미스 제어	장비 레시피, 결함 데이터, 사내 소스 코드 등 민감 자산의 외부 클라우드(OpenAI 등) 유출 원천 차단 요구.	망 분리 환경에서 구동 가능한 vLLM 기반 로컬 모델(LLaMA 3 등) 배포 및 Model Context Protocol (MCP)을 통한 접근 권한 최소화 설계[13].
결정론적 검증 **(Deterministic)**	AI의 환각(Hallucination) 현상에 의한 잘못된 장비 제어나 코딩 오류가 대규모 물량 폐기 및 인프라 장애로 직결될 위험.	단순 프롬프트 체이닝을 배제하고, LangGraph 기반의 상태 머신(State Machine)을 도입하여 모든 에이전트 행동을 명시적이고 결정론적인 루프로 통제[15].
데이터 추적성 **(Traceability)**	AI가 수율 하락 원인을 지목하거나 패치를 제안했을 때, 어떤 근거(문서, 로그)를 바탕으로 도출했는지 엔지니어가 검증할 수 있어야 함.	RAG(Retrieval-Augmented Generation) 쿼리 시 대상 SECS/GEM 로그의 타임스탬프와 매뉴얼 페이지를 명시하고, NeMo Guardrails를 통한 환각 차단[6].
격리된 실행 샌드박스	AI 에이전트가 코드를 수정하고 자율적으로 스크립트를 실행할 때 운영(Production) 환경에 예기치 않은 부작용(Side Effect)을 초래할 위험.	Open-SWE 아키텍처를 차용하여, 모든 코드 수정 및 테스트 실행을 Daytona 또는 컨테이너 기반의 클라우드 샌드박스 내부로 완벽히 격리[12].

2. 2025~2026 글로벌 AI 해커톤 우승작 패턴 심층 분석

최근 2년간 글로벌 탑티어 테크 기업(NVIDIA, AWS, GitLab, Kong, MIT/Stanford)이 주최하거나 Devpost 상위 1%에 랭크된 AI 해커톤 수상작들의 아키텍처를 분석해 보면, 단일 LLM에 의존하는 챗봇(Chatbot)이나 단순 API 래퍼(Wrapper) 형태의 프로젝트는 완전히 자취를

감추었다[19]. 현대 해커톤의 절대적인 성공 방정식은 여러 개의 특화된 에이전트가 복잡한 인프라와 직접 소통하며 자율적으로 문제를 해결하는 '에이전틱 워크플로우(Agentic Workflow)'의 완벽한 구현에 있다.

LangGraph 중심의 결정론적 상태 머신**(State Machine)** 오케스트레이션

과거 다중 에이전트 시스템(Multi-Agent System)의 한계는 확률적으로 텍스트를 생성하는 LLM의 특성상 에이전트 간의 대화가 무한 루프에 빠지거나 궤도를 이탈하는 현상이었다. 최근의 압도적 우승작들은 LangChain에서 개발한 LangGraph 프레임워크를 채택하여 이 문제를 근본적으로 해결했다[15]. LangGraph는 에이전트의 워크플로우를 유향 순환 그래프(Directed Cyclic Graph, DCG) 형태의 '결정론적 상태 머신(Deterministic State Machine)'으로 모델링한다[21].

이는 에이전트가 단순히 자유롭게 대화하는 것이 아니라, 명확하게 정의된 노드(Node)와 간선(Edge)을 따라 이동하며 각 단계마다 반드시 검증 모듈을 거치도록 강제하는 아키텍처다. 예를 들어, 코드를 작성하는 노드(Programmer)가 결과물을 내놓으면 반드시 평가 노드(Reviewer)로 라우팅되어 테스트를 수행하며, 실패 시 에러 로그와 함께 다시 작성 노드로 되돌아가는 명시적이고 상태가 유지되는(Stateful) 재시도 루프를 형성한다[18]. 이러한 중앙 집중식 상태 관리(Shared State)는 엔터프라이즈 환경에서 필수적인 예측 가능성과 감사 가능성(Auditability)을 보장한다[25].

도구 실행**(Tool Use)**의 진화**: Model Context Protocol (MCP)** 통합

에이전트가 기업 내부의 데이터베이스나 GitHub, Jira 등의 시스템과 안전하게 통신하기 위한 프로토콜로 Anthropic이 주도하는 Model Context Protocol (MCP)이 해커톤의 표준 기술로 자리 잡았다[13]. 과거에는 개발자가 직접 LLM에 개별 API 엔드포인트를 하드코딩하여 부여했다면, 이제는 MCP 서버를 통해 기업 인프라에 대한 접근을 추상화하고 보안 계층을 분리한다.

대표적으로 2025년 Kong Agentic AI Hackathon에서 2위를 차지한 'AgenticAI-MCP-client' 프로젝트는 이 기술의 정수를 보여주었다. 이 팀은 Kong API Gateway 내부에 MCP 클라이언트를 통합하여 Mongo MCP 서버와 연결했다. 에이전트가 자연어 입력을 받아 구조화된 시스템 쿼리로 변환하여 실행할 때, 모든 통신이 API 게이트웨이의 강력한 보안 및 관측성(Observability) 제어를 받도록 설계함으로써 공격 표면(Attack Surface)을 획기적으로 줄이고 운영 오버헤드를 감소시킨 점이 심사위원들의 극찬을 받았다[13].

개발자 생산성**(SWE)** 및 인프라 자가 치유**(Self-Healing)**의 프로덕션화

해커톤에서 가장 높은 평가를 받는 도메인은 단연 '개발자 생산성 향상'과 '시스템 장애 대응 자동화'이다. 특히 SWE-agent, Devin 등의 등장을 기점으로, 해커톤 우승작들은 단순한 코드 추천을 넘어 실제 런타임 환경에서 시스템을 진단하고 자가 복구하는 수준으로 진화했다[27]. 최근 LangChain이 발표한 Open-SWE 프레임워크는 Stripe, Coinbase 등에서 내부적으로 구축했던 코딩 에이전트의 구조를 표준화한 것으로, Manager, Planner, Programmer, Reviewer로 이루어진 서브 에이전트 아키텍처와 컨테이너 샌드박스 격리 패턴을 완벽히 정립했다[12].

실제 글로벌 해커톤 우승 사례를 살펴보면 이러한 아키텍처적 진화가 뚜렷하게 확인된다.

- **AWS Global Hackathon 2025** 우승 **(Self-healing AI infrastructure):** 분산 시스템 엔지니어인 Shantanu가 구현한 이 프로젝트는 인프라 런타임 에러를 EventBridge로 감지한 후, 에이전트가 스스로 코드를 분석하여 패치(Mutation)를 생성하고 프로덕션 환경에 핫스왑(Hot-swap)으로 무중단 배포하는 시스템이다. 사용자 채팅을 처리하는 층과 모델을 훈련·패치하는 진화 루프(Evolution Loop)를 완전히 분리하여 가동

중단(Downtime)을 제로화한 점이 우승의 결정적 요인이었다[29].

- **GitLab AI Hackathon 2026** 대상 **(LORE):** 총상금 65,000달러 규모의 이 대회에서 우승한 LORE 시스템은 8개의 전문화된 에이전트와 지식 그래프를 결합했다. 라우터(Router) 모듈이 질문의 성격에 맞춰 적절한 에이전트에게 작업을 할당하고, 지식 그래프 내부의 순환 논리 오류(Circular loop)를 방지하는 철저한 로직을 구현했다. 특히 코드의 환경적 영향을 분석하는 탄소 발자국 추적(Carbon tracking) 기능을 통합하여 지속가능성(Sustainability) 지표까지 포괄하는 기업가적 마인드를 증명했다[30].
- **MIT Media Lab NandaHack 2026** 우승 **(AgentPress):** 뉴스룸 에이전시를 모사한 이 시스템은 에이전트들이 스스로 정보를 수집하고 기사를 작성하면, 'Herald'라는 편집장 에이전트가 8가지의 결정론적 검증 루브릭(Deterministic 8-factor rubric)에 기반하여 결과물을 평가하고 반려하는 상호 검증 구조를 설계하여 1위를 차지했다[31].

3. 심사위원(기업 테크 리더/임원)의 평가 심리 및 채점 기준 분해

기업 기술 임원과 수석 엔지니어들로 구성된 해커톤 심사위원단은 화려한 프론트엔드 UI 이면에 숨겨진 백엔드의 '실제 작동 여부(Feasibility)'를 간파하는 데 매우 능숙하다. 대상을 거머쥐기 위해서는 심사위원들이 마음속으로 채점하는 가점과 감점의 심리적 트리거를 역설계(Reverse-engineering)해야 한다.

즉시 탈락을 유발하는 **Hard Reject** 기준

기술 심사위원들이 프로젝트를 '흔한 토이 프로젝트'로 치부하며 평가 대상에서 제외하는 패턴은 명확하다.

1. 제어되지 않은 환각 현상 **(Unhandled Hallucination):** 에이전트가 존재하지 않는 장비 알람 코드를 지어내거나 인프라 복구 쉘(Shell) 명령어를 임의로 생성하는 등의 환각을 방치한 설계는 엔터프라이즈 환경에서 치명적인 장애를 유발한다. NeMo Guardrails나 LangGraph의 Fallback 라우팅과 같은 체계적인 방어막이 없는 프로젝트는 즉시 탈락한다[17].
2. 슬라이드웨어 및 정적 목업**(Mock-up)** 데모**:** 실제 런타임 샌드박스에서 코드가 컴파일되거나 DB 쿼리가 도는 것이 아니라, 사전에 하드코딩된 시나리오대로 버튼을 누르면 그럴듯한 텍스트가 출력되는 프론트엔드 위주의 목업 프로젝트는 혹독한 평가를 받는다[19].
3. 비현실적인 **API** 호출 낭비 및 지연 시간**(Latency):** 복잡한 에이전트 추론을 구현했다 하더라도, 결과 도출을 위해 수십 번의 순차적(Sequential) LLM API 호출을 발생시켜 한 번의 응답에 1~2분이 소요되도록 방치한 구조는 실효성이 없다. 비동기 처리나 스트리밍 로직이 부재한 파이프라인은 현장 도입 불가 판정을 받는다[10].

마음을 사로잡는 **High Score** 트리거 **(WOW Factor 3**가지**)**

반면, 심사위원의 탄성을 자아내고 1등을 확정 짓는 핵심 요소는 다음 세 가지로 요약된다.

1. 실질적인 비즈니스 임팩트의 수치화 **(ROI** 및 공수 절감**):** 추상적인 '업무 효율성 증대'가 아니라, 기존 공정의 소요 시간과 비용을 얼마나 정량적으로 단축했는지 증명해야 한다. 최근 학계에 발표된 'SemiFA' 논문은 반도체 수율 저하 분석에 소요되던 수 시간의 전문가 수작업(Manual FA) 파이프라인을 다중 모달 에이전트로 대체하여 단 48초(150~300배 속도 향상)로 단축했음을 정량적으로 증명하여 압도적인 임팩트를 남겼다[8].

2. 프로덕션 레벨의 신뢰성 **(**검증 파이프라인 및 **Human-in-the-loop):** 완벽한 AI란 없다는 것을 쿨하게 인정하고, 에이전트의 확신도(Confidence score)가 임계치 이하로 떨어지거나 코딩 컴파일 오류가 발생할 경우, 자의적으로 판단하지 않고 LangGraph의 interrupt before 노드를 통해 즉시 현업 엔지니어의 슬랙(Slack)으로 승인 요청을 보내는 'Human-in-the-loop' 폴백(Fallback) 전략이 구현되어야 한다[18].
3. 백엔드 아키텍처의 가시성 **(Observability):** 프론트엔드 앱의 완성도보다, 에이전트가 샌드박스 환경 내부에서 터미널 쉘을 열고 직접 grep이나 pytest 명령어를 입력하며 에러를 탐색하는 생생한 런타임 과정이나, LangGraph Studio를 통해 실시간으로 상태 노드(Graph State)가 라우팅되는 트래픽 시각화 화면을 라이브 데모로 보여줄 때 심사위원들은 열광한다[18].

4. 1등 수상 타겟 출품작 기획안 (최소 3개 제안)

해커톤 제한 시간 내에 구현이 가능하면서도, SK하이닉스 기술 임원진의 최대 고민거리인 반도체 양산 수율 관리, MLOps 자가 치유, 그리고 현장 엔지니어의 지식 검색 병목을 완벽하게 해결할 수 있는 에이전틱 워크플로우 기반 청사진 3가지를 제안한다.

제안 **1: Project [Yield-Sentinel] - SECS/GEM** 다중 모달 융합 기반 수율 장애 자율 분석 에이전트

1. 프로젝트명 및 한 줄 정의**:** 반도체 장비 SECS/GEM 시계열 통신 로그와 웨이퍼 결함 이미지를 다중 모달(Multi-modal)로 융합 추론하여 8D 문제 해결 리포트를 1분 내에 자동 생성하는 에이전틱 수율 방어 시스템[8].
2. 풀고자 하는 핵심 문제**(Pain Point):** HBM 패키징 등 고난도 공정에서 수율 저하(Yield Excursion)가 발생할 때, FDC 장비 알람 데이터와 광학 검사(AOI) 이미지 간의 컨텍스트가 단절되어 있어 엔지니어가 원인 장비와 챔버를 추적하는 데 막대한 시간이 낭비되는 병목 현상[5].
3. **End-to-End** 시스템 아키텍처 **(SemiFA** 프레임워크 응용**):**
   - 데이터 수집**:** 설비의 SECS/GEM 로그(HSMS 프로토콜 시뮬레이터 활용) 및 KLA 장비 사양의 웨이퍼 결함 패턴 이미지(WM-811K 등 오픈 데이터셋 활용)를 Kafka 이벤트 스트림으로 인제스트[8].
   - 에이전트 오케스트레이션 **(LangGraph** 상태 머신**):**
     - *DefectDescriber* 노드*:* ResNet-50 대비 파라미터는 적으면서 미세 패치 분류에 압도적 성능을 보이는 DINOv2 시각 인코더와 LLaVA-1.6 모델을 활용해 결함의 형태학적 특성을 텍스트로 서술[8].
     - *RootCauseAnalyzer* 노드*:* 추출된 SECS/GEM 온도/압력 피처와 결함 서술 텍스트를 융합하여, 과거 유사 수율 저하 사례를 Qdrant 벡터 DB에서 RAG로 검색 후 원인 장비 추론[8].
   - 검증 및 도구 실행**:** 추론된 원인과 레시피 조정 권고안을 포함한 구조화된 PDF 형태의 8D 리포트를 조립하여 Slack 채널로 발송 및 Jira 티켓 생성[6].
4. 차별화된 기술적 엣지**(Edge):** 단순한 비전(Vision) 기반의 이미지 분류를 넘어, 도메인 특화 데이터인 시계열 설비 로그(SECS/GEM)를 텍스트 시퀀스로 변환하여 대형 시각언어모델(VLM)의 프롬프트 컨텍스트에 융합(Multi-modal fusion)하는 산업계 최초의 자동화 파이프라인 구현[10].
5. 해커톤 라이브 데모 시나리오 **(3**분**):**

- 스크린 좌측에 웨이퍼 결함 맵(D3.js 시각화)과 우측에 쉴 새 없이 스크롤되는 가상의 장비 로그 터미널을 띄운다.
- 라이브로 Out-of-Control 알람을 트리거하면, Yield-Sentinel의 LangGraph 노드들이 순차적으로 점등하며 48초 만에 백그라운드 분석을 마친다.
- 심사위원의 눈앞에서 DINOv2가 결함을 특정 짓고, 융합 모델이 "식각 장비 3번 챔버의 압력 밸브 미세 누출"이라는 정확한 Root Cause를 도출한 PDF 리포트가 슬랙에 팝업되는 장면으로 마무리[10].

6. 예상 심사위원 **Q&A** 및 방어 논리**:**

- *Q:* 팹*(Fab)* 내부의 *SECS/GEM* 로그는 외부 클라우드로 전송이 불가능한데 실효성이 있는가*?*
- *A:* 본 아키텍처는 외부 *API* 의존성을 배제하고 모든 모델*(LLaVA, DINOv2)*과 벡터 *DB(Qdrant)*가 로컬 컨테이너 환경*(On-premise)*에서 작동하도록 설계되어 데이터 유출 리스크가 원천 차단됩니다.

제안 **2: Project [HBM-AutoFix] - Open-SWE** 기반 테스트 인프라 **SRE** 자가 치유 파이프라인

1. 프로젝트명 및 한 줄 정의**:** HBM 테스트 및 패키징 장비를 구동하는 MLOps/DevOps 파이프라인에서 런타임 스크립트 에러 발생 시, 에러 로그 분석부터 코드 격리 패치 및 PR(Pull Request) 생성까지 자율 수행하는 SRE 자가 치유 시스템[12].
2. 풀고자 하는 핵심 문제**(Pain Point):** 수백 대의 장비와 서버를 통제하는 자동화 스크립트에 사소한 구문 오류(Syntax error)나 종속성(Dependency) 충돌이 발생할 경우, 전체 라인의 병목을 유발하며 엔지니어가 일일이 터미널에 접속해 핫픽스를 배포해야 하는 리소스 낭비[6].
3. **End-to-End** 시스템 아키텍처 **(Open-SWE** 프레임워크 응용**):**

- 데이터 수집**:** GitLab CI/CD 파이프라인에서 에러 반환(Stderr) 시 Webhook을 통해 에러 컨텍스트와 스택 트레이스 수집.
- 에이전트 오케스트레이션 **(LangGraph & Daytona Sandbox):**
  - 워크스페이스 루트의 AGENTS.md 파일을 파싱하여 SK하이닉스 사내 코딩 컨벤션 및 안전 수칙을 시스템 프롬프트에 주입(Context Engineering)[12].
  - *Manager & Planner:* 에러 원인을 분석하고 단계별 디버깅 계획 수립.
  - *Programmer:* Daytona 기반의 완전히 격리된 클라우드 샌드박스 내부에서 쉘(Shell) 명령어를 실행하여 코드를 직접 수정[12].
  - *Reviewer:* 수정된 코드를 대상으로 pytest 등의 유닛 테스트를 실행. 실패 시 에러 로그를 Planner에게 피드백하여 상태 머신 내에서 무결성이 확보될 때까지 자동 재시도 루프 수행[18].
- 도구 실행**:** 테스트 성공 시 GitHub/GitLab에 "Fix: HBM Test Script Timeout" 제목으로 Pull Request를 오픈하고 리뷰어 할당[18].

4. 차별화된 기술적 엣지**(Edge):** 개발자의 코딩을 돕는 수동적인 '코파일럿' 수준을 넘어, 에이전트가 격리된 샌드박스 컨테이너의 터미널 통제권을 쥐고 자율적으로 시스템 환경을 탐색(grep, ls)하고 수정 사항을 검증하는 진정한 의미의 'Agentic SRE'를 선도적으로 적용[12].
5. 해커톤 라이브 데모 시나리오 **(3**분**):**

- 시연자가 반도체 테스트 장비를 모사한 파이썬 스크립트에 의도적으로 타임아웃 오류(Time-out error) 로직을 주입하여 빌드를 고장 낸다.
- HBM-AutoFix 에이전트가 즉각 호출되어 샌드박스 내부로 진입한다.
- 청중은 터미널 창을 통해 에이전트가 grep으로 에러 발생 라인을 찾고, sed 명령어 혹은 파이썬 스크립트 도구를 사용해 코드를 수정한 뒤 빌드를 재실행하여 초록색(Pass) 로그를 뽑아내는 숨막히는 과정을 실시간으로 관전한다.
- 깔끔하게 정리된 PR 페이지가 브라우저에 팝업되며 종료[12].

6. 예상 심사위원 **Q&A** 및 방어 논리**:**

- *Q: AI*가 무작위로 코드를 수정하다가 멀쩡한 다른 공정 제어 로직까지 망가뜨릴*(Side Effect)* 위험은 어떻게 통제하는가*?*
- A: 프로덕션 환경과의 물리적 격리(Daytona Sandbox)가 최우선으로 적용되며, 수정된 코드는 사전에 정의된 결정론적 유닛 테스트를 통과하지 않으면 절대 PR로 승격되지 않습니다. 최종 반영 버튼 역시 시니어 엔지니어의 리뷰(Human-in-the-loop)를 거치도록 설계되었습니다[12].

제안 **3: Project [Nexus-Fab] - MCP** 기반 분산 인프라 지식 검색 및 제어 플랫폼

1. 프로젝트명 및 한 줄 정의**:** 사내에 파편화된 MES(제조 실행 시스템), FDC 센서, 장비 매뉴얼, ERP 재고 DB를 Model Context Protocol(MCP)로 단일화하여, 엔지니어의 자연어 질의만으로 공정 제어 및 부품 교체 워크오더를 자동 발행하는 지능형 에이전트 플랫폼[6].
2. 풀고자 하는 핵심 문제**(Pain Point):** 장비 이상 발생 시 엔지니어가 FDC 대시보드에서 센서 트렌드를 확인하고, 수백 페이지의 PDF 장비 매뉴얼을 뒤진 후, 다시 ERP 시스템에 접속해 수리 부품 재고를 조회하는 전형적인 분절 업무(Swivel-chair process)에 따른 비효율성[5].
3. **End-to-End** 시스템 아키텍처**:**

- 데이터 통합 인프라 **(Kong API Gateway + MCP Server):** 보안이 취약한 직접 DB 연결 방식을 탈피하여, 사내 장비 매뉴얼(Qdrant 벡터 DB), SECS/GEM 이력 DB, ERP 재고 시스템 각각을 독립된 MCP 서버 인스턴스로 감싸고 Kong API Gateway를 통해 통신[13].
- 에이전트 오케스트레이션 **(Router Agent):** 사용자가 슬랙 채널에서 자연어로 질의를 던지면, 라우터 에이전트가 질의의 의도를 분석하여 적합한 MCP 도구(Tool)로 컨텍스트 쿼리를 동적으로 분배[13].
- 도구 실행**:** 다중 소스에서 정보를 취합한 뒤, 이상 원인을 분석하고 즉시 ERP MCP 도구를 호출해 O-Ring 등 필요 부품의 출고 티켓을 발행[6].

4. 차별화된 기술적 엣지**(Edge):** 엔터프라이즈 환경에서 보안 부서가 가장 우려하는 '사내망 DB 접근 권한 부여' 문제를, Kong의 API 게이트웨이와 MCP의 표준화된 보안 계층 설계를 통해 선제적으로 방어하여 아키텍처의 프로덕션 적합성(Production Readiness)을 극대화[13].
5. 해커톤 라이브 데모 시나리오 **(3**분**):**

- 슬랙(Slack) 채팅창에서 "오늘 새벽 HBM 라인 3번 장비의 식각 챔버 압력 저하 원인 매뉴얼에서 찾아 요약하고, 관련 펌프 부품 재고 파악해서 워크오더(Work-order) 생성해줘"라고 입력.
- 시스템 백그라운드 트레이스 로그가 투명하게 화면에 노출되며, 에이전트가 1) FDC 로그

DB 조회 ➡️ 2) Qdrant 매뉴얼 RAG 검색 ➡️ 3) ERP 재고 확인 ➡️ 4) Jira 티켓 발행을 릴레이로 수행.
    - 단 한 줄의 프롬프트가 복합적인 엔터프라이즈 트랜잭션으로 매끄럽게 변환되는 통합 시연[6].
6. 예상 심사위원 **Q&A** 및 방어 논리**:**
    - *Q:* 에이전트가 환각*(Hallucination)*에 빠져 엉뚱한 값비싼 부품을 대량으로 출고 처리해버린다면 어떻게 대처하는가*?*
    - A: 읽기 전용(Read-only) 쿼리와 상태 변경(State-mutating) 트랜잭션에 대한 권한 계층을 엄격히 분리했습니다. 부품 출고와 같은 쓰기 작업은 LangGraph의 interrupt before 노드에 걸려, 슬랙 메시지로 전달된 버튼을 관리자가 직접 클릭(Human-in-the-loop)해야만 최종 승인 API가 트리거되도록 안전 장치를 마련했습니다[18].

5. 최종 우승을 위한 데모 & 피칭 플레이북

해커톤의 우승은 시스템의 코드 라인 수가 아니라, 제한된 시간 내에 심사위원을 단숨에 설득하는 스토리텔링과 무대 장악력에서 결정된다. 피칭의 핵심은 '우리의 기술이 얼마나 신기한가'를 뽐내는 것이 아니라, '현장의 막대한 손실을 어떻게 틀어막을 것인가'를 증명하는 것이다.

슬라이드 구성 및 스토리텔링 프레임워크 **(30 - 30 - 40** 법칙**)**

심사위원이 지루할 틈을 주지 않는 3분 압축 피칭 프레임워크는 다음과 같다.

- 도입부 **(**문제 정의 및 비즈니스 임팩트**, 30%):** 가상의 문제가 아니라 SK하이닉스 기술 임원진이 실적 발표나 학회에서 언급한 실제 병목(예: HBM 20단 적층 공정의 극악의 난이도, 방대한 센서 데이터 분석의 한계)을 인용하여 시작한다[3]. 스피치 예시*:* "현재 *HBM* 패키징 라인에서 수율 분석을 위한 *8D* 리포트 작성에 숙련된 엔지니어가 평균 *3*시간을 소모합니다. 우리는 *LangGraph*와 다중 모달 융합 기술을 통해 이를 단 *48*초로 단축했습니다."[10]
- 중반부 **(**기술 아키텍처 및 차별성 증명**, 30%):** 복잡하고 난해한 코드 조각을 나열하지 말고, **End-to-End** 시스템 아키텍처 다이어그램 하나로 승부한다. 단순히 "LLM을 썼습니다"가 아니라, "데이터가 어디서 인제스트되고(Kafka), 어떤 안전 샌드박스를 거치며(Daytona), 어떻게 결정론적 루프로 통제되어(LangGraph), 사내 인프라(Jira/ERP)로 매핑되는지"를 명확한 흐름으로 설명한다[14].
- 종반부 **(**라이브 시연 및 확장성 증명**, 40%):** 슬라이드 쇼를 과감히 종료하고 실제 터미널 또는 UI로 전환하여 실시간 동작(Live Demo)을 시연한다. 에이전트가 생각하고 행동하는 상태 이동(Graph State Routing) 과정을 LangGraph Studio 화면이나 라이브 터미널 뷰를 통해 명시적으로 보여주어, 시스템이 블랙박스가 아님을 증명한다[19].

실행 가능성**(Feasibility)** 증명을 위한 정량적 지표 설계법

우승을 확정 짓기 위해서는 심사위원이 경영진에게 "이 프로젝트를 당장 사내에 도입합시다"라고 보고할 수 있는 '정량적 ROI 지표'를 데모 대시보드에 명확히 시각화하여 제시해야 한다.

핵심 평가 지표 **(Metrics)**	설명 및 피칭 어필 포인트

MTTR (Mean Time To Recovery) 단축률	인프라 장애나 수율 저하 감지부터 원인 파악 및 코드 패치/복구까지 걸리는 시간을 기존 수작업 대비 몇 배 단축했는지 증명 (예: "매뉴얼 분석 4시간 ➡️ 에이전트 분석 48초, 약 300배 향상")[8].
에이전트 자가 복구 성공률 **(Self-healing Rate)**	격리된 샌드박스 내부에서 발생한 구문 오류(Syntax Error)를 단일 상태 머신 피드백 루프 내에서 스스로 인지하고 교정에 성공한 횟수 및 비율 표출[12].
결함 검출 정확도 **(F1-Score / mAP)**	웨이퍼 이미지 비전 모델로 ResNet-50과 같은 무거운 레거시를 쓰지 않고, 파라미터는 110배 적으면서 성능은 7.1%p 더 높은 DINOv2를 채택하여 추론 비용을 극적으로 낮췄음을 증명 (Accuracy 92.1%, Macro F1 0.917)[8].
LLM 토큰 효율성 및 실행 비용 **(Cost per Task)**	가장 비싸고 거대한 API(GPT-4o 등)에 무조건 의존하는 대신, 라우팅 작업은 소형 모델(Llama-3 8B)로 처리하고 복잡한 추론만 선택적으로 위임하여 1건의 워크오더 처리 비용을 $0.05 이하로 최적화한 엔터프라이즈 감각 어필.

결론적으로 [SK하이닉스 AI 해커톤 2026]에서의 압도적인 대상 수상은 최근 AI 트렌드를 '텍스트 생성'에서 '인프라와 상호작용하는 결정론적 시스템 제어'로 패러다임 전환을 이뤄낼 수 있느냐에 달려있다. 심사위원단은 단순한 아이디어의 나열이 아니라, HBM 양산 현장의 극심한 수율 압박과 방대한 SECS/GEM 설비 데이터의 가공이라는 가장 치열한 병목을 직접 타격하는 프로덕션 레벨의 해결책을 기다리고 있다. 본 보고서에서 제안한 LangGraph, MCP, Open-SWE 기반의 3가지 에이전틱 워크플로우 기획안을 정밀하게 구현하여 해커톤 무대에 올린다면, 압도적인 기술적 격차로 우승의 영광을 차지할 수 있을 것이다.

참고 자료

1. Five things you need to know about SK hynix and the future of AI, https://news.skhynix.com/en/5-things-about-skhynix/
2. Why advanced memory matters in the AI infrastructure era, https://news.skhynix.com/en/ai-competition-next-challenge/
3. SK hynix holds firm on 90% yield threshold for hybrid bonding, https://mbiz.heraldcorp.com/article/10878775
4. Semiconductor Acronyms Glossary: Chip & Equipment Terms, https://umbrex.com/resources/industry-acronym-glossaries/technology-telecom

-industry-acronyms/semiconductors-semiconductor-equipment-industry-acronyms/
5. High-Quality Data Needed To Better Utilize Fab Data Streams, https://semiengineering.com/high-data-quality-needed-to-better-utilize-fab-data-streams/
6. AI for Semiconductor — Yield Analytics & Equipment Intelligence, https://www.zealogics.com/industries/semiconductor
7. Protocol support - runZero, https://help.runzero.com/docs/troubleshooting-protocols-supported/
8. SemiFA: An Agentic Multi-Modal Framework for Autonomous ... - arXiv, https://arxiv.org/pdf/2604.13236
9. Yield Management Embraces Expanding Role, https://semiengineering.com/yield-management-embraces-expanding-role/
10. SemiFA: An Agentic Multi-Modal Framework for Autonomous ... - arXiv, https://arxiv.org/html/2604.13236v1
11. Getting Smarter About Tool Maintenance - Semiconductor Engineering, https://semiengineering.com/getting-smarter-about-tool-maintenance/
12. Open SWE: LangChain's Open-Source Coding Agent Framework for, https://www.tuncer-byte.com/en/blog/open-swe-framework
13. Announcing the Winners of Kong Agentic AI Hackathon 2025, https://konghq.com/blog/news/winners-of-kong-agentic-ai-hackathon
14. SemiFA: Agentic Multi-Modal Framework for Autonomous ... - GitHub, https://github.com/Shivamckaushik/SemiFA
15. LLM-Based Multi-Agent Orchestration: A Survey of Frameworks, https://www.preprints.org/manuscript/202604.2147
16. AutoGen vs LangGraph — Conversation Agents or State Graphs, https://myengineeringpath.dev/tools/autogen-vs-langgraph/
17. Guardrails AI and NVIDIA NeMo Guardrails - My Framer Site, https://guardrailsai.com/blog/nemoguardrails-integration
18. LangChain Launches Open SWE, an Open-Source Asynchronous, https://www.infoq.com/news/2025/08/langchain-open-swe/
19. AI Hackathon Ideas for 2026: 20 Projects That Use Agentic, https://blogs.reskilll.com/ai-hackathon-ideas-2026-agentic-ai-mcp-projects/
20. 3 Years of Graph Engineering with LangGraph - LangChain, https://www.langchain.com/blog/3-years-of-graph-engineering-with-langgraph
21. Rêve | State Management in Multi-Agent Systems using LangGraph, https://www.reve.cloud/blog/state-management-in-multi-agent-systems-using-langgraph-a-technical-deep-dive
22. LangGraph Multi-Agent Architecture: Building a Self-Critiquing AI, https://towardsai.com/p/machine-learning/langgraph-multi-agent-architecture-building-a-self-critiquing-ai-debate-system
23. Building Self-Correcting AI Agent Workflows with LangGraph, https://www.axontick.com/blogs/stateful-retries-evaluation-nodes-langgraph
24. LangGraph in Production: Choosing and Building Multi-Agent Systems, https://subratpati.medium.com/langgraph-in-production-choosing-and-building-

multi-agent-systems-c2b955f16429
25. Multi-Agent Framework Communication Protocols, https://www.tdcommons.org/cgi/viewcontent.cgi?filename=6&article=10298&context=dpubs_series&type=additional
26. GitHub - ai-boost/awesome-harness-engineering, https://github.com/ai-boost/awesome-harness-engineering
27. Saarthi: The First AI Formal Verification Engineer - arXiv, https://arxiv.org/html/2502.16662v2
28. Agentic AI for Software: thoughts from Software Engineering ... - arXiv, https://arxiv.org/html/2508.17343v3
29. Shantanu Awate Shantanu-00 - GitHub, https://github.com/Shantanu-00
30. GitLab AI Hackathon 2026: Meet the winners, https://about.gitlab.com/blog/gitlab-ai-hackathon-2026-meet-the-winners/
31. NandaHack: Agentic AI Hackathon · HCLTech and MIT Media Lab, https://nandahack.media.mit.edu/
32. Agentic AI Hackathon (2026) - Startup Grants India, https://www.startupgrantsindia.com/competitions/agentic-ai-hackathon-5
33. Silicon Wafer Engineering | Atomic Loops, https://www.atomicloops.com/industries/silicon-wafer-engineering
34. Agentic workflows for end-to-end software engineering automation, https://wjarr.com/sites/default/files/fulltext_pdf/WJARR-2025-0887.pdf
35. Open SWE Captures the Architecture That Stripe, Coinbase and, https://devops.com/open-swe-captures-the-architecture-that-stripe-coinbase-and-ramp-built-independently-for-internal-coding-agents/